

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN THIẾT KẾ LUẬN LÝ

BÁO CÁO ĐỒ ÁN

Hệ thống IoT RTOS giám sát chất lượng không khí (IAQ)

Học kỳ 2, năm học 2025–2026

Giảng viên hướng dẫn Phan Văn Sỹ
Lớp L02

Sinh viên	MSSV
Hồ Công Danh	2310410
Hoàng Anh Duy	2310458
Cao Quang Đôn	2310752

TP. Hồ Chí Minh, tháng 05 năm 2026

Tóm tắt

Đề tài xây dựng một hệ thống IoT RTOS giám sát và điều chỉnh chất lượng không khí trong nhà (IAQ). Hệ thống gồm thiết bị nhúng ESP32, firmware đọc cảm biến, tầng RTOS/IoT truyền dữ liệu, backend server, cơ sở dữ liệu và dashboard web.

Phần firmware hiện tại đọc bụi mịn PMS7003 qua UART, CO2 và nhiệt độ/độ ẩm qua RS485 Modbus, VOC qua ADC, sau đó chuẩn hóa dữ liệu thành **SensorSample**. Dữ liệu được đánh giá bởi **IaqEvaluator** theo vùng an toàn/cần can thiệp/cảnh báo, áp dụng hysteresis và hold-time để giảm dao động, rồi điều khiển LED hoặc thiết bị mô phỏng qua **IaqController**. Hệ thống cũng ghi CSV vào SPIFFS để phục vụ kiểm thử.

Các phần RTOS, MQTT, backend, database và dashboard được thiết kế theo hướng mở rộng: ESP32 publish telemetry lên MQTT, backend nhận dữ liệu và cung cấp API, database lưu lịch sử, dashboard hiển thị realtime/history và gửi lệnh điều khiển ngược xuống thiết bị.

Mục lục

Tóm tắt	i
1 Giới thiệu đề tài	1
1.1 Bối cảnh	1
1.2 Mục tiêu	1
1.3 Phạm vi	2
2 Yêu cầu và cơ sở lý thuyết	3
2.1 Các đại lượng giám sát IAQ	3
2.2 Yêu cầu chức năng	3
2.3 Yêu cầu phi chức năng	4
3 Kiến trúc hệ thống	5
3.1 Luồng dữ liệu	5
3.2 Phân rã module	6
3.3 Nguyên tắc thiết kế	6
4 Thiết kế phần cứng	7
4.1 Nền tảng điều khiển	7
4.2 Cảm biến và giao tiếp	7
4.3 Thiết bị chấp hành và cảnh báo	8
5 Firmware đọc dữ liệu và điều khiển cục bộ	9
5.1 Mẫu dữ liệu thống nhất	9
5.2 Đọc cảm biến	9
5.3 Đánh giá IAQ	9
5.4 Log và vận hành cục bộ	10
6 Thiết kế RTOS và IoT truyền dữ liệu	11
6.1 Thiết kế RTOS	11
6.2 Thiết kế MQTT	11
6.3 Cơ chế vận hành khi mất kết nối	12

7	Server, cơ sở dữ liệu và dashboard web	13
7.1	Backend server	13
7.2	Cơ sở dữ liệu	13
7.3	Dashboard web	14
7.4	Luồng điều khiển web đến thiết bị	14
8	Kiểm thử và đánh giá	15
8.1	Kiểm thử firmware	15
8.2	Kiểm thử tích hợp	15
8.3	Chỉ số đánh giá	16
9	Kết luận và hướng phát triển	17
	Tài liệu tham khảo	18

1. Giới thiệu đề tài

Đề tài xây dựng một hệ thống IoT RTOS giám sát và điều chỉnh chất lượng không khí trong nhà (Indoor Air Quality – IAQ). Hệ thống dùng ESP32 làm nút nhúng trung tâm, thu thập dữ liệu từ nhiều loại cảm biến, đánh giá trạng thái môi trường theo thời gian thực, điều khiển thiết bị chấp hành cục bộ và định hướng gửi dữ liệu lên server để hiển thị trên dashboard web.

Trong phạm vi source hiện tại, phần firmware phần cứng đã được hiện thực rõ nhất. Firmware đọc PMS7003 qua UART, đọc CO₂ và nhiệt độ/độ ẩm qua RS485 Modbus, đọc VOC qua ADC, đóng gói thành `SensorSample`, đánh giá IAQ bằng `IaqEvaluator`, điều khiển LED/thiết bị mô phỏng qua `IaqController`, và ghi log CSV vào SPIFFS bằng `CsvLogger`. Các phần RTOS, MQTT, backend, database và frontend đang được định nghĩa theo module README và contract chung để mở rộng thành hệ thống hoàn chỉnh.

1.1 Bối cảnh

Phòng học và phòng làm việc kín thường có nguy cơ tích tụ CO₂, bụi mịn, VOC, nhiệt độ cao hoặc độ ẩm không phù hợp. Các yếu tố này ảnh hưởng đến sức khỏe, sự tập trung và mức độ thoải mái của người sử dụng. Việc giám sát thủ công không đủ liên tục, trong khi hệ thống IoT có thể đo, cảnh báo và điều khiển tự động theo trạng thái thực tế.

1.2 Mục tiêu

- Thiết kế phần cứng đo IAQ dựa trên ESP32 và các cảm biến phổ biến.
- Viết firmware đọc dữ liệu đa giao thức, chuẩn hóa mẫu đo và xử lý lỗi cảm biến.
- Thiết kế tầng RTOS để tách các chức năng đọc cảm biến, xử lý, điều khiển, log và truyền MQTT.
- Thiết kế luồng IoT gửi dữ liệu từ ESP32 lên backend, lưu dữ liệu lịch sử và cung cấp API cho dashboard.

- Thiết kế dashboard web theo dõi realtime, xem lịch sử và gửi lệnh điều khiển thiết bị.

1.3 Phạm vi

Báo cáo tập trung vào thiết kế tổng thể và hiện trạng source trong repository. Phần hardware/firmware được mô tả ở mức hiện thực. Phần RTOS, IoT, server, database và dashboard được trình bày theo thiết kế module, contract dữ liệu và hướng tích hợp vì trong repo hiện chủ yếu có README định hướng cho các phần này.

2. Yêu cầu và cơ sở lý thuyết

2.1 Các đại lượng giám sát IAQ

Hệ thống theo dõi năm nhóm đại lượng chính: CO₂, PM_{2.5}/PM₁₀, VOC, nhiệt độ và độ ẩm. CO₂ phản ánh mức thông gió và mật độ người trong phòng. PM_{2.5}/PM₁₀ phản ánh bụi mịn. VOC phản ánh hợp chất hữu cơ bay hơi từ vật liệu, hóa chất hoặc hoạt động sinh hoạt. Nhiệt độ và độ ẩm ảnh hưởng đến tiện nghi nhiệt và cảm nhận môi trường.

Bảng 2.1: Mốc vận hành tham khảo cho hệ thống IAQ

Chỉ số	An toàn	Cần can thiệp	Cảnh báo
CO ₂	Dưới 1000 ppm	1000–1500 ppm	Trên 1500 ppm
PM _{2.5}	Dưới 15 $\mu g/m^3$	15–35 $\mu g/m^3$	Trên 35 $\mu g/m^3$
VOC	Dưới 500	500–1000	Trên 1000
Nhiệt độ	Dưới 28 °C	28–32 °C	Trên 32 °C
Độ ẩm	30–50%RH	25–30%RH hoặc 50–60%RH	Dưới 25%RH hoặc trên 60%RH

Các mốc trên được dùng cho prototype điều khiển. Khi triển khai thật, nhóm cần hiệu chuẩn lại theo cảm biến thực tế, điều kiện phòng và tiêu chuẩn vận hành được chọn.

2.2 Yêu cầu chức năng

- ESP32 đọc dữ liệu cảm biến theo chu kỳ và tạo mẫu đo có dấu thời gian.
- Firmware biểu diễn được dữ liệu thiếu hoặc lỗi bằng cờ hợp lệ, tránh dùng giá trị sai cho quyết định điều khiển.
- Hệ thống phân vùng từng chỉ số thành **SAFE**, **INTERVENE** và **ALARM**.
- Điều khiển thiết bị cục bộ theo nguyên nhân: thông gió cho CO₂/VOC, HEPA cho bụi mịn, than hoạt tính cho VOC, điều hòa cho nhiệt độ, tạo ẩm cho độ ẩm thấp.
- Ghi log CSV cục bộ để kiểm thử và đối chiếu.

6. Gửi dữ liệu lên backend qua MQTT trong phiên bản IoT hoàn chỉnh.
7. Dashboard hiển thị realtime, lịch sử và hỗ trợ gửi lệnh điều khiển.

2.3 Yêu cầu phi chức năng

Firmware cần nhẹ, ổn định trên ESP32 và tránh làm treo vòng lặp khi cảm biến mất kết nối. Các driver có timeout, dữ liệu số thực được ưu tiên lưu dạng fixed-point x10, và tầng điều khiển dùng hysteresis/hold-time để giảm bật tắt liên tục. Phần server cần validate dữ liệu MQTT/API, tách cấu hình qua biến môi trường và lưu lịch sử theo timestamp để truy vấn dashboard.

3. Kiến trúc hệ thống

Kiến trúc tổng thể được chia thành bốn khối: thiết bị nhúng, truyền thông IoT, server/backend và dashboard web. Thiết bị nhúng chịu trách nhiệm đo và phản ứng cục bộ. Server chịu trách nhiệm nhận dữ liệu, lưu lịch sử và cung cấp API. Dashboard web là giao diện quan sát và điều khiển.

3.1 Luồng dữ liệu

Sensor -> ESP32/Firmware -> MQTT -> Backend -> Database -> API -> Web
Dashboard

Luồng điều khiển đi ngược lại:

Web Dashboard -> API -> Backend -> MQTT -> ESP32 -> Device/LED

Trong source hiện tại, đoạn Sensor -> ESP32/Firmware -> log/LED đã được hiện thực. Các đoạn MQTT, backend, database và web đang được mô tả qua README module và contract chung.

3.2 Phân rã module

Bảng 3.1: Các module chính của dự án

Module	Trách nhiệm	Hiện trạng trong repo
hardware/main	Driver cảm biến, đánh giá IAQ, điều khiển output, ghi log SPIFFS	Có source C++/Arduino
rtos	Thiết kế task, queue, timing và MQTT client trên ESP32	Có README định hướng
common	MQTT topic, data format, API contract	Có README contract
iot_backend/backend	Nhận MQTT, lưu dữ liệu, cung cấp API	Có README thiết kế
iot_backend/database	Lưu dữ liệu sensor theo thời gian	Có README thiết kế MySQL
iot_backend/frontend	Dashboard realtime, history, control panel	Có README thiết kế
integration_test	Kiểm thử pipeline dữ liệu và điều khiển end-to-end	Có README test plan

3.3 Nguyên tắc thiết kế

Các tầng được tách theo trách nhiệm. Driver chỉ đọc cảm biến và kiểm tra giao thức. **SensorHub** gom dữ liệu thành **SensorSample**. **IaqEvaluator** chỉ xử lý trạng thái IAQ, không phụ thuộc chân GPIO. **IaqController** chỉ áp dụng output. Backend và dashboard không đọc trực tiếp phần cứng, mà làm việc qua MQTT/API.

4. Thiết kế phần cứng

4.1 Nền tảng điều khiển

Thiết bị nhúng dùng ESP32 NodeMCU-32S. ESP32 phù hợp vì có nhiều UART phần cứng, ADC, Wi-Fi tích hợp và hỗ trợ FreeRTOS. Trong firmware hiện tại, UART1 được dùng cho PMS7003, UART2 được dùng cho RS485 Modbus, và ADC1 GPIO34 được dùng cho VOC analog.

Bảng 4.1: Pin map chính trong firmware

Chức năng	Pin ESP32	Ghi chú
PMS RX	GPIO 26	ESP32 nhận dữ liệu từ TX của PMS7003.
PMS TX	GPIO 27	Dự phòng nếu cần gửi lệnh cho PMS7003.
RS485 RX	GPIO 16	UART2 nhận dữ liệu từ module RS485.
RS485 TX	GPIO 17	UART2 gửi request Modbus RTU.
RS485 DE/RE	-1	Source giả định module RS485 tự điều hướng truyền/nhận.
VOC ADC	GPIO 34	ADC1 input-only, phù hợp đọc tín hiệu analog.

4.2 Cảm biến và giao tiếp

PMS7003 truyền frame nhị phân qua UART 9600 bps. Driver tìm header 0x42 0x4D, đọc đủ frame, kiểm tra checksum rồi tách các giá trị PM1.0, PM2.5 và PM10.

CO2 và SHTC3 dùng RS485 Modbus RTU. Firmware tạo request function code 0x03, tính CRC16, gửi qua UART2 và kiểm tra response trước khi dùng dữ liệu. CO2 được đọc từ slave ID 1, register 0x0001; SHTC3 được đọc từ slave ID 2, hai register bắt đầu tại 0x0000.

VOC được đọc qua ADC 12-bit. Lớp `VocAnalog` cấu hình ADC attenuation và dùng buffer vòng để tính trung bình trượt, giúp giảm nhiễu tức thời.

4.3 Thiết bị chấp hành và cảnh báo

Source hiện tại dùng LED để mô phỏng thiết bị chấp hành và cảnh báo. LED xanh biểu diễn thiết bị như HEPA, thông gió, lọc than, điều hòa, tạo ẩm. LED đỏ biểu diễn cảnh báo theo từng nguyên nhân: CO₂, PM, VOC, nhiệt độ và độ ẩm. Các chân được định nghĩa trong `Pins.h` và có thể đổi theo wiring thực tế.

5. Firmware đọc dữ liệu và điều khiển cục bộ

5.1 Mẫu dữ liệu thống nhất

Firmware dùng `SensorSample` làm cấu trúc dữ liệu trung tâm. Mẫu đo chứa thời gian `ms`, các chỉ số PM, nhiệt độ x10, độ ẩm x10, CO2 ppm, VOC raw, VOC trung bình và cờ hợp lệ cho từng nhóm cảm biến. Các giá trị thiếu được mã hóa bằng sentinel như -1, INT16_MIN hoặc 0xFFFF.

Listing 5.1: Luồng xử lý chính trong loop

```
SensorSample s = g_hub.readSample();  
g_store.publish(s);  
  
g_iaq = g_eval.evaluate(s);  
g_ctrl.apply(g_iaq);  
  
printIAQ(s, g_iaq);  
g_csv.logToSpiffs(s);
```

5.2 Đọc cảm biến

`SensorHub` là lớp gom dữ liệu từ các driver. Khi gọi `readSample()`, hub đọc PMS7003, đọc register CO2, đọc register nhiệt độ/độ ẩm và đọc VOC analog. Nếu một cảm biến không trả dữ liệu đúng, các cờ `ok_pms`, `ok_co2`, `ok_sht` giữ giá trị false để tầng sau không hiểu nhầm dữ liệu lỗi là dữ liệu thật.

5.3 Đánh giá IAQ

`IaqEvaluator` ánh xạ dữ liệu đo sang `IaqState`. Mỗi chỉ số có vùng SAFE, INTERVENE, ALARM. Với CO2, PM và VOC, hàm `eval3` dùng ngưỡng an toàn, ngưỡng cảnh báo và

hysteresis. Với nhiệt độ và độ ẩm, firmware có hàm riêng vì trạng thái tốt là nằm trong khoảng vận hành.

Sau khi phân vùng, evaluator sinh yêu cầu điều khiển: CO₂/VOC kích hoạt thông gió, PM kích hoạt HEPA, VOC kích hoạt lọc than, nhiệt độ kích hoạt điều hòa, độ ẩm thấp kích hoạt tạo ẩm. Các output dùng hold-time 5000 ms để tránh bật/tắt liên tục.

5.4 Log và vận hành cục bộ

CsvLogger ghi dữ liệu vào SPIFFS tại `/log.csv`. Firmware hỗ trợ lệnh Serial **START**, **STOP**, **CLEAR**, **DUMP**, **INTERVAL=...** để phục vụ thí nghiệm. Nhờ đó nhóm có thể thu dữ liệu trước khi hoàn thiện phần MQTT/backend.

6. Thiết kế RTOS và IoT truyền dữ liệu

6.1 Thiết kế RTOS

Phiên bản firmware hiện tại chạy tuần tự trong Arduino `loop()`. Khi chuyển sang RTOS, các chức năng nên tách thành task để đảm bảo timing rõ ràng và tránh một thao tác I/O làm chậm toàn bộ hệ thống.

Bảng 6.1: Đề xuất phân chia task RTOS

Task	Chức năng	Dữ liệu trao đổi
Task_Sensor	Đọc PMS7003, Modbus, VOC theo chu kỳ	Gửi <code>SensorSample</code> vào queue/store
Task_Process	Đánh giá IAQ, hysteresis, hold-time	Nhận <code>SensorSample</code> , xuất <code>IaqState</code>
Task_Control	Cập nhật LED/thiết bị chấp hành	Nhận <code>IaqState</code>
Task_MQTT	Publish telemetry và nhận lệnh điều khiển	MQTT payload JSON
Task_Log	Ghi CSV cục bộ hoặc dump log	<code>SensorSample</code> theo chu kỳ

`SnapshotStore` trong source hiện tại đã dùng critical section để bảo vệ mẫu đo mới nhất. Đây là nền tảng phù hợp khi nhiều task cùng đọc/ghi dữ liệu.

6.2 Thiết kế MQTT

Theo contract chung, hệ thống dùng các topic:

- `iot/sensor/data`: ESP32 publish telemetry lên backend.
- `iot/device/control`: backend publish lệnh điều khiển xuống ESP32.

Payload dữ liệu cần chứa `device_id`, timestamp, dữ liệu cảm biến và cờ trạng thái. Với firmware hiện tại, payload nên mở rộng từ `SensorSample` để có đủ PM, CO2, VOC, nhiệt độ, độ ẩm và cờ hợp lệ thay vì chỉ gửi temperature/humidity.

6.3 Cơ chế vận hành khi mất kết nối

ESP32 không nên phụ thuộc hoàn toàn vào server. Khi MQTT hoặc Wi-Fi mất kết nối, thiết bị vẫn đọc cảm biến, đánh giá IAQ, điều khiển cục bộ và ghi log SPIFFS. Khi kết nối khôi phục, hệ thống có thể tiếp tục publish dữ liệu realtime; phần đồng bộ log lịch sử là hướng mở rộng.

7. Server, cơ sở dữ liệu và dashboard web

7.1 Backend server

Backend được thiết kế bằng Node.js và Express. Vai trò chính là nhận dữ liệu từ MQTT broker, validate payload, lưu dữ liệu lịch sử vào database và cung cấp API cho dashboard. README backend định hướng các endpoint chính: GET /api/data, GET /api/history, POST /api/control.

Bảng 7.1: API dự kiến cho dashboard

API	Chức năng	Nguồn/đích dữ liệu
GET /api/data	Lấy trạng thái realtime mới nhất	RAM/cache backend
GET /api/history	Lấy dữ liệu lịch sử theo khoảng thời gian	Database
POST /api/control	Nhận lệnh từ web và publish MQTT xuống ESP32	MQTT topic control

7.2 Cơ sở dữ liệu

README database đang mô tả MySQL để lưu dữ liệu sensor theo thời gian. Bảng dữ liệu cần có `device_id`, `timestamp`, các chỉ số IAQ và trạng thái hợp lệ. Với dữ liệu chuỗi thời gian, cột `created_at` hoặc `timestamp` cần được index để truy vấn lịch sử nhanh.

Realtime không nên lấy trực tiếp từ database. Backend nên giữ mẫu mới nhất trong RAM/cache để dashboard gọi nhanh qua /api/data; database phục vụ biểu đồ lịch sử, thống kê và phân tích sau.

7.3 Dashboard web

Frontend được thiết kế bằng HTML/CSS/JavaScript, Bootstrap và Chart.js. Dashboard cần có ba nhóm màn hình: realtime dashboard, analytics/history và control panel. Realtime dashboard hiển thị CO₂, PM_{2.5}, VOC, nhiệt độ, độ ẩm, vùng IAQ và trạng thái thiết bị. Analytics hiển thị biểu đồ theo thời gian. Control panel gửi lệnh bật/tắt hoặc override qua API.

7.4 Luồng điều khiển web đến thiết bị

Khi người dùng nhấn nút điều khiển trên dashboard, frontend gọi `POST /api/control`. Backend validate lệnh, publish xuống `iot/device/control`. ESP32 nhận MQTT message, cập nhật trạng thái điều khiển hoặc override theo chính sách an toàn. Các lệnh từ web không nên bỏ qua cảnh báo cục bộ; firmware vẫn cần ưu tiên trạng thái **ALARM**.

8. Kiểm thử và đánh giá

8.1 Kiểm thử firmware

Kiểm thử firmware tập trung vào từng driver và luồng xử lý chính. Các thư mục test hiện có gồm `testPMS7003`, `testCO2`, `testSHTC3`, `testVOC`. Mỗi test cần xác nhận cảm biến trả dữ liệu hợp lệ, timeout không làm treo hệ thống và cờ hợp lệ phản ánh đúng trạng thái kết nối.

Bảng 8.1: Kế hoạch kiểm thử firmware

Nhóm kiểm thử	Mục tiêu	Kết quả kỳ vọng
PMS7003	Kiểm tra UART frame và checksum	<code>ok_pms=1</code> , PM thay đổi hợp lý
Modbus CO2	Kiểm tra request/response RS485	Đọc được ppm, timeout khi rút cảm biến
SHTC3 RS485	Kiểm tra nhiệt độ, độ ẩm fixed-point	Giá trị đúng đơn vị $^{\circ}C$, %RH
VOC ADC	Kiểm tra ADC và trung bình trượt	Raw thay đổi, avg mượt hơn raw trượt
IAQ evaluator	Kiểm tra zone và output	Đổi trạng thái đúng ngưỡng, có hysteresis
CSV logger	Kiểm tra ghi/dump/clear log	File CSV đủ header và dữ liệu

8.2 Kiểm thử tích hợp

Khi hoàn thiện IoT/backend/frontend, kiểm thử tích hợp cần bao phủ hai pipeline:

- Data pipeline: Sensor -> ESP32 -> MQTT -> Backend -> DB -> API -> Web.
- Control pipeline: Web -> API -> Backend -> MQTT -> ESP32 -> Device.

README integration test đã định hướng hai script `test_pipeline.js` và `test_con`

`trol_flow.js`. Hai script này nên kiểm tra dữ liệu có đi đến dashboard và lệnh từ web có làm thay đổi LED/thiết bị đúng như kỳ vọng.

8.3 Chỉ số đánh giá

Các chỉ số đánh giá gồm tỷ lệ mẫu đo hợp lệ, độ trễ từ lúc vượt ngưỡng đến lúc output đổi trạng thái, số lần bật/tắt trong một khoảng thời gian, tỷ lệ thời gian ở vùng an toàn, khả năng vận hành khi mất server và độ ổn định của pipeline MQTT/API/web.

9. Kết luận và hướng phát triển

Báo cáo đã tổ chức lại dự án theo đúng các phần của một hệ thống IoT RTOS giám sát và điều chỉnh chất lượng không khí: thiết kế phần cứng, firmware đọc dữ liệu, RTOS/IoT truyền dữ liệu, server/database và dashboard web. Source hiện tại đã có nền tảng firmware quan trọng để đọc cảm biến, đánh giá IAQ, điều khiển output và ghi log cục bộ.

Hướng phát triển trực tiếp là hiện thực tầng RTOS bằng các task và queue, bổ sung MQTT client trên ESP32, chuẩn hóa payload JSON theo `common`, xây dựng backend Node.js/Express nhận MQTT và cung cấp API, thiết kế database lưu lịch sử sensor, sau đó hoàn thiện dashboard web realtime/history/control.

Khi hệ thống chạy end-to-end, nhóm cần thu dữ liệu thực tế trong phòng học hoặc phòng làm việc, hiệu chuẩn ngưỡng theo môi trường thật và bổ sung cơ chế xử lý lỗi như mất Wi-Fi, mất MQTT, cảm biến timeout, payload sai định dạng hoặc database không khả dụng.

Tài liệu tham khảo

- [1] Nhóm đồ án, *Source code firmware ESP32 trong thư mục IOT_RTOS_Project/hardware/main*, repository CO3091_DA_TKLL_252.
- [2] Nhóm đồ án, *RTOS Module README*, thư mục IOT_RTOS_Project/rtos.
- [3] Nhóm đồ án, *Backend Module README*, thư mục IOT_RTOS_Project/iot_backend/backend.
- [4] Nhóm đồ án, *Frontend Module README*, thư mục IOT_RTOS_Project/iot_backend/frontend.
- [5] Nhóm đồ án, *Common Contract README*, thư mục IOT_RTOS_Project/common.
- [6] Tài liệu tham khảo trong repository, *Cuộc thi KHKT năm 2025–2026: Hệ thống giám sát và điều chỉnh chất lượng không khí trong phòng học sử dụng nền tảng E-Ra IoT*.
- [7] Espressif Systems, *Arduino core for the ESP32*, tài liệu lập trình ESP32 với Arduino framework.
- [8] Modbus Organization, *MODBUS Application Protocol Specification*, tài liệu giao thức Modbus.
- [9] Plantower, *PMS7003 Digital Universal Particle Concentration Sensor Datasheet*.